

Small atomistic jobs can leave a GPU underused. Batching runs several together; replacing completed jobs keeps useful work in the batch.

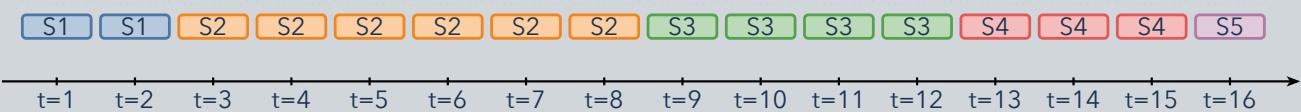
1 Compare scheduling strategies

Ten structures needing 2, 6, 4, 3, 6, 5, 2, 4, 3, 2 ionic steps, on 5 GPU slots

Unbatched Simulations

20% slots busy

37 steps to finish all 10

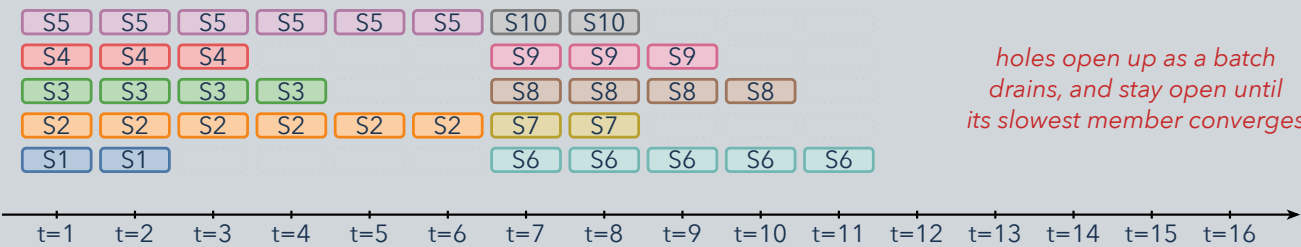


...21 more steps

Binning AutoBatcher

67% slots busy

11 steps to finish all 10



holes open up as a batch drains, and stay open until its slowest member converges

InFlight AutoBatcher

93% slots busy

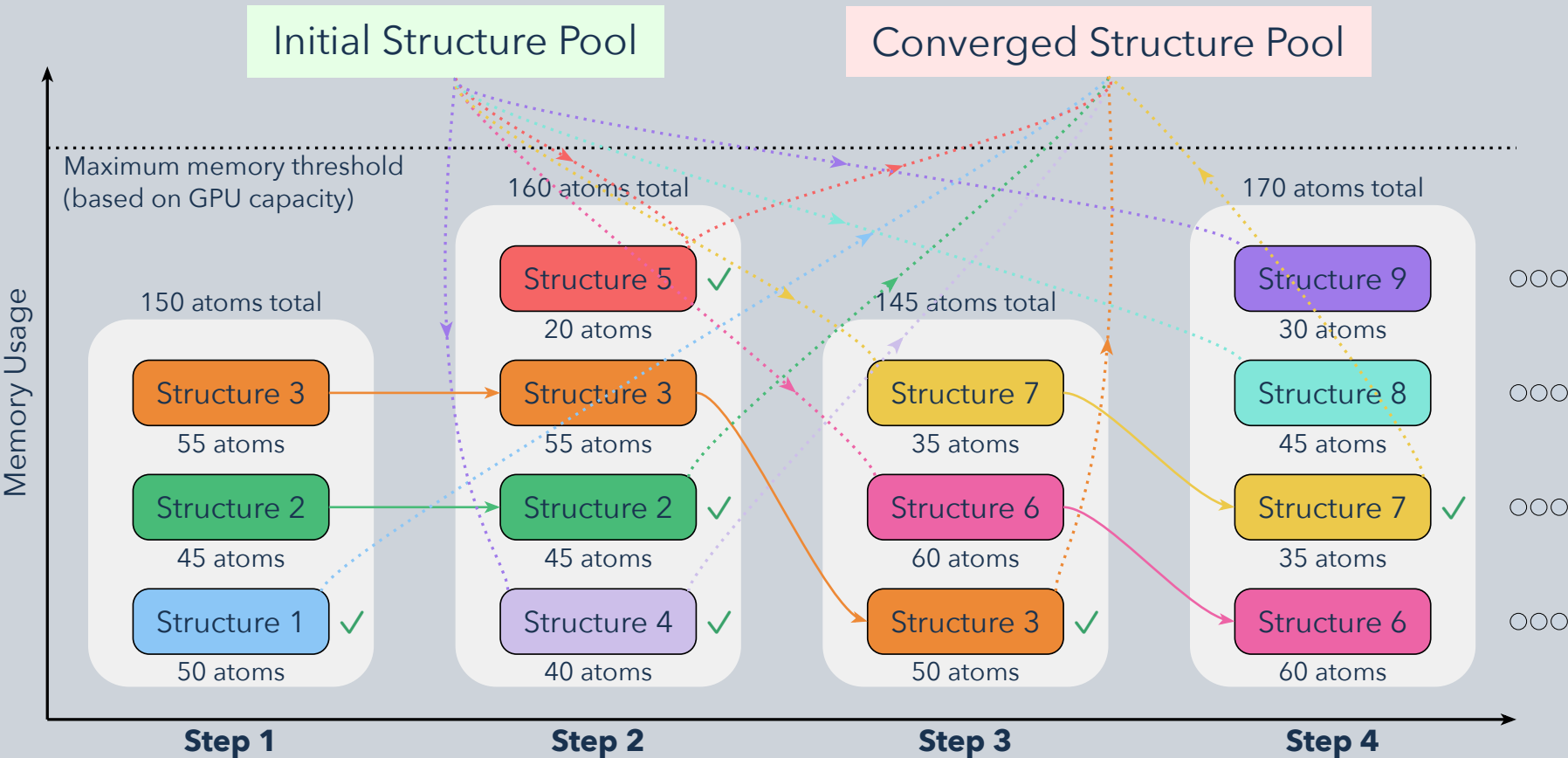
8 steps to finish all 10



all ten converged by t=8

Unbatched leaves four of five slots idle. **Binning** holds each slot until the batch’s slowest member converges. **In-flight** immediately reuses freed slots: a color change within a row marks a new structure taking over.

2 Follow a replacement



Each structure has its own convergence state. After a structure finishes, move it to the completed pool and fill the available capacity from the waiting pool.

Batch by resource use, not just job count. Atom counts, neighbor counts, memory, and model cost constrain admission. Utilization percentages in the schematic are illustrative, not measured benchmarks.